Introduction to Databases and SQL

What is a Database?

A **database** is an organized collection of data that is stored electronically and can be easily accessed, managed, updated, and retrieved.

In simple terms, a database is like a digital filing cabinet where information is stored in a structured manner.

Real-Life Examples

- Student Management System
- Library Management System
- Hospital Records System
- Banking System
- E-Commerce Websites
- Social Media Platforms

Example

A college database may store:

Student ID	Name	Course	Marks
101	Rahul	Computer Science	85
102	Priya	Information Technology	90
103	Amit	Mechanical	78

Why Do We Need Databases?

Before databases, information was often stored in files such as spreadsheets or text documents. As data grew larger, managing it became difficult.

Problems with Traditional File Systems

1. Data Redundancy (duplicate data)
2. Data Inconsistency
3. Difficulty in searching records
4. Poor security
5. Limited data sharing
6. Difficult backup and recovery

Advantages of Databases

- ✅ Organized storage of data
- ✅ Fast data retrieval
- ✅ Reduced duplication
- ✅ Better security
- ✅ Data sharing among multiple users
- ✅ Backup and recovery support
- ✅ Data integrity and consistency

Database Management System (DBMS)

A **Database Management System (DBMS)** is software that allows users to create, manage, and interact with databases.

Functions of a DBMS

- Create databases
- Store data
- Update records
- Delete records
- Search information
- Manage security
- Handle backups

Popular DBMS Software

DBMS	Description
MySQL	Open-source relational database
PostgreSQL	Advanced open-source database
Oracle Database	Enterprise-level DBMS
Microsoft SQL Server	Microsoft's database solution
SQLite	Lightweight embedded database
MongoDB	NoSQL database

Types of Databases

1. Relational Database

Stores data in tables consisting of rows and columns.

Examples

- MySQL
- PostgreSQL
- Oracle
- SQL Server

Example:

Students Table

StudentID	Name	Course
101	Rahul	CS
102	Priya	IT

---

2. NoSQL Database

Stores data in flexible formats such as documents, key-value pairs, or graphs.

Examples

- MongoDB
- Cassandra
- Redis

Example:

```
{
  "StudentID": 101,
  "Name": "Rahul",
  "Course": "CS"
}
```

Understanding Tables, Rows, and Columns

Table

A table stores related data.

Example:

Employees Table

EmployeeID	Name	Department
1	John	HR
2	Alice	IT

Row (Record)

A row represents a single entry.

Example:

1 | John | HR

This entire line is one record.

Column (Field)

A column represents a specific attribute.

Example:

EmployeeID
Name
Department

These are columns.

What is SQL?

SQL (Structured Query Language) is the standard language used to communicate with relational databases.

SQL allows users to:

- Create databases
- Create tables
- Insert data
- Update data
- Delete data

- Retrieve data
- Manage permissions

Features of SQL

Simple Language

Easy to learn and use.

Standardized

Used by almost all relational databases.

Powerful

Can handle large amounts of data.

Portable

Works across different database systems.

SQL Categories

SQL commands are grouped into several categories.

---

1. DDL (Data Definition Language)

Used to define database structures.

Commands

- CREATE
- ALTER
- DROP
- TRUNCATE

Example

```
CREATE TABLE Students (
   StudentID INT,
   Name VARCHAR(50),
   Course VARCHAR(50)
);
```

---

2. DML (Data Manipulation Language)

Used to manipulate data inside tables.

Commands

- INSERT
- UPDATE
- DELETE

Example

```
INSERT INTO Students
VALUES (101, 'Rahul', 'CS');
```

---

3. DQL (Data Query Language)

Used to retrieve data.

Command

- SELECT

Example:

```
SELECT * FROM Students;
```

---

4. DCL (Data Control Language)

Used to control access and permissions.

Commands

- GRANT
- REVOKE

Example:

```
GRANT SELECT ON Students TO User1;
```

---

5. TCL (Transaction Control Language)

Used to manage transactions.

Commands

- COMMIT
- ROLLBACK
- SAVEPOINT

Example:

COMMIT;

Creating a Database

SQL Syntax

CREATE DATABASE CollegeDB;

This creates a new database named CollegeDB.

Using a Database

USE CollegeDB;

This selects the database for use.

Creating Tables

Example:

```
CREATE TABLE Students (
   StudentID INT PRIMARY KEY,
   Name VARCHAR(100),
   Age INT,
   Course VARCHAR(50)
);
```

Data Types in SQL

Numeric Types

Data Type	Description
INT	Integer values
FLOAT	Decimal values
DOUBLE	Large decimal values

Example:

Age INT

Character Types

Data Type	Description
CHAR	Fixed length text
VARCHAR	Variable length text
TEXT	Large text

Example:

Name VARCHAR(100)

Date and Time Types

Data Type	Description
DATE	Date
TIME	Time
DATETIME	Date and time

Example:

JoiningDate DATE

Inserting Data

Syntax

```
INSERT INTO Students
(StudentID, Name, Age, Course)
VALUES
(101, 'Rahul', 20, 'CS');
```

Multiple Records

```
INSERT INTO Students
VALUES
(102, 'Priya', 21, 'IT'),
(103, 'Amit', 19, 'Mechanical');
```

Retrieving Data

Display All Records

```
SELECT * FROM Students;
```

Display Specific Columns

```
SELECT Name, Course
FROM Students;
```

Filtering Data with WHERE

Example

```
SELECT *
FROM Students
WHERE Age > 20;
```

Multiple Conditions

```
SELECT *
FROM Students
WHERE Age > 18 AND Course = 'CS';
```

Updating Records

Syntax

```
UPDATE Students
SET Age = 22
WHERE StudentID = 101;
```

Deleting Records

Syntax

```
DELETE FROM Students
WHERE StudentID = 101;
```

Sorting Data

Ascending Order

```
SELECT *
FROM Students
ORDER BY Name ASC;
```

Descending Order

```
SELECT *
FROM Students
ORDER BY Marks DESC;
```

SQL Constraints

Constraints enforce rules on data.

PRIMARY KEY

Uniquely identifies each record.

StudentID INT PRIMARY KEY

NOT NULL

Prevents empty values.

Name VARCHAR(100) NOT NULL

UNIQUE

Ensures unique values.

Email VARCHAR(100) UNIQUE

DEFAULT

Provides default values.

Country VARCHAR(50) DEFAULT 'India'

Primary Key vs Foreign Key

Primary Key

Uniquely identifies records.

Example:

StudentID

Foreign Key

Links one table to another.

Students Table

StudentID	Name
101	Rahul

Results Table

ResultID	StudentID	Marks
1	101	85

Here:

StudentID in Results

is a Foreign Key referencing Students.

Joins in SQL

Joins combine data from multiple tables.

INNER JOIN

Returns matching records.

```
SELECT Students.Name,
    Results.Marks
FROM Students
INNER JOIN Results
ON Students.StudentID = Results.StudentID;
```

LEFT JOIN

Returns all records from the left table and matching records from the right table.

```
SELECT *
FROM Students
LEFT JOIN Results
ON Students.StudentID = Results.StudentID;
```

Aggregate Functions

Used for calculations on multiple rows.

COUNT()

```
SELECT COUNT(*)
FROM Students;
```

SUM()

```
SELECT SUM(Marks)
FROM Results;
```

AVG()

```
SELECT AVG(Marks)
FROM Results;
```

MAX()

```
SELECT MAX(Marks)
FROM Results;
```

MIN()

```
SELECT MIN(Marks)
FROM Results;
```

SQL Best Practices

1. Use meaningful table names.
2. Define primary keys.
3. Avoid duplicate data.
4. Use constraints where appropriate.
5. Backup databases regularly.
6. Write readable SQL queries.
7. Use indexes for large datasets.
8. Follow consistent naming conventions.

Common SQL Applications

Education

- Student Management Systems
- Examination Systems

Banking

- Account Management
- Transactions

Healthcare

- Patient Records
- Appointment Systems

E-Commerce

- Product Catalogs
- Orders
- Payments

Social Media

- User Profiles
- Posts
- Comments

Summary

A **Database** is a structured collection of data, while a **DBMS** is software used to manage that data. **SQL (Structured Query Language)** is the standard language used to create, manage, and retrieve information from relational databases. Understanding databases, tables, keys, constraints, and basic SQL operations is essential for anyone starting a career in Data Science, Software Development, Data Analytics, Web Development, or Database Administration.